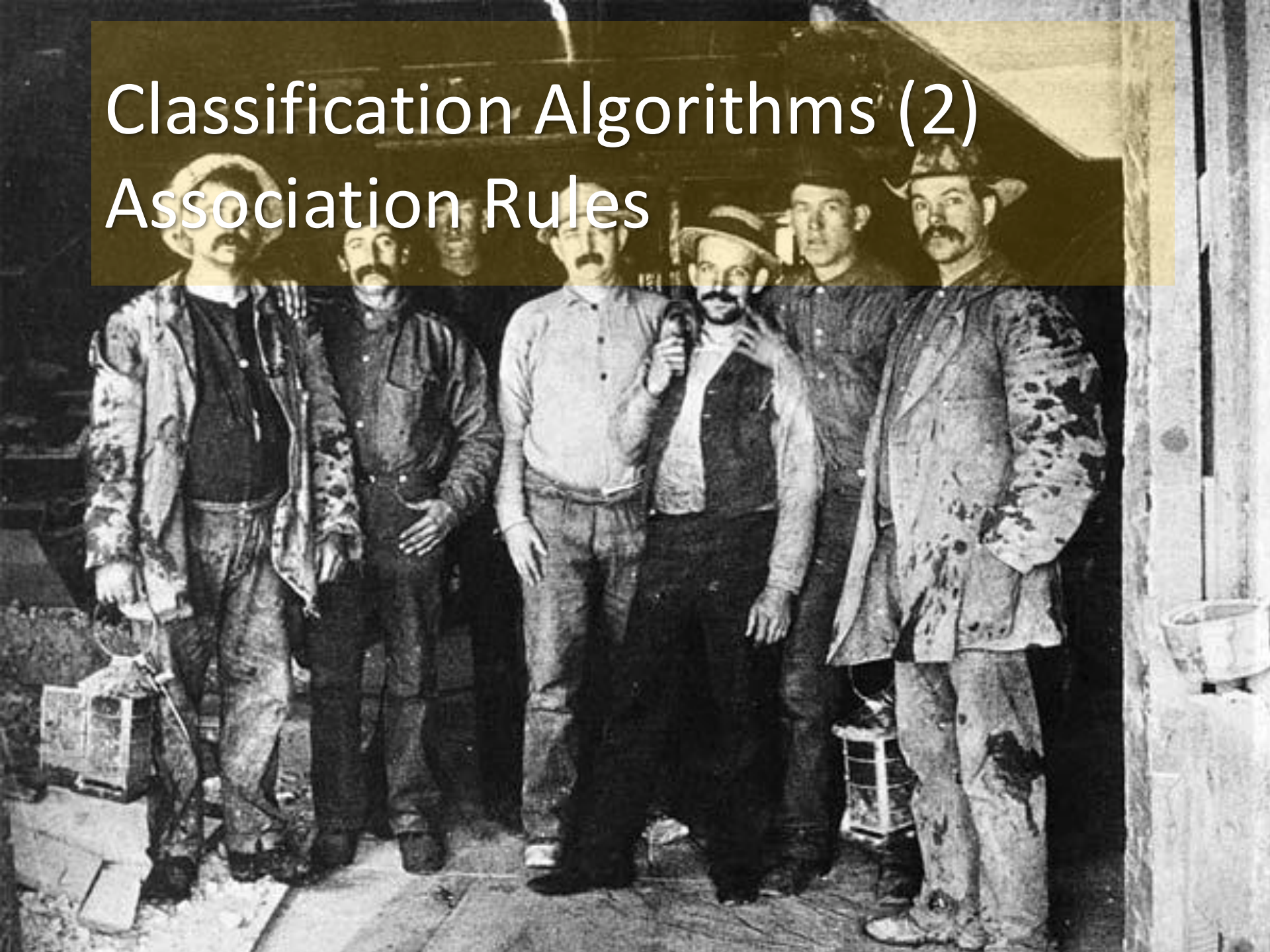


Classification Algorithms (2)

Association Rules



The Laplace estimator



- Problem
 - Given the data if one of $P(A_i | \text{yes}) = 0$
 - then $P(\text{yes} | A_1, \dots, A_k) = 0$!
- Solution
 - outlook has 3 possible values, so if we would add 3 artificial cases to our dataset we would get:
$$P(\text{outlook}=\text{sunny} | \text{yes}) = (0 + 1) / (5 + 3) = 0.125$$

Instance-based learning

- Other names

- **Case-based learning**

- “FOUR MEN LINKED TO THE POPULAR PIRATE BAY FILE-SHARING SITE WERE *JAILED...*” ON 17/04/2009. (mirror.co.uk). If you run a file-sharing site, you better be scared!

- **Memory-based learning**

- You “remember” the data and don’t build a model on it

- **Non-parametric learning**

- If you remember the data, there is no need to approximate parameters (e.g., slope of a linear function)

Nearest neighbour classifier

Given a training set of vectors $x_1, \dots, x_n$ in $\mathbf{R}^m$ with their classifications (labels) $y_1, \dots, y_n$ in $\{0,1\}$ (or in $\{0, \dots, K\}$).

TASK: classify (assign labels to) vectors from a test set

$$x'_1, x'_2, x'_3, \dots$$

Main Idea:

“cases that are *close/similar* to each other should have the same labels”

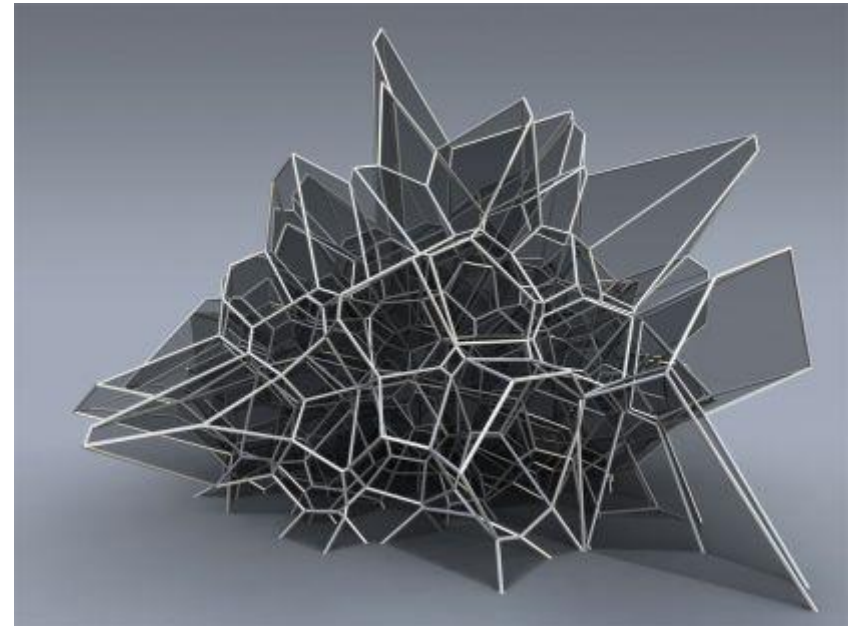
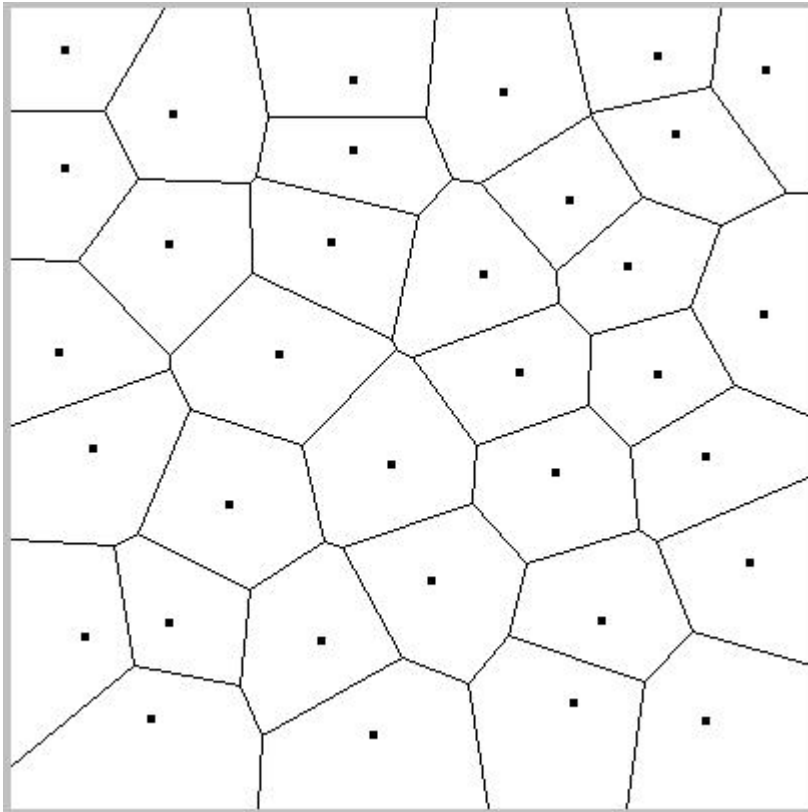


To find a label for the x'

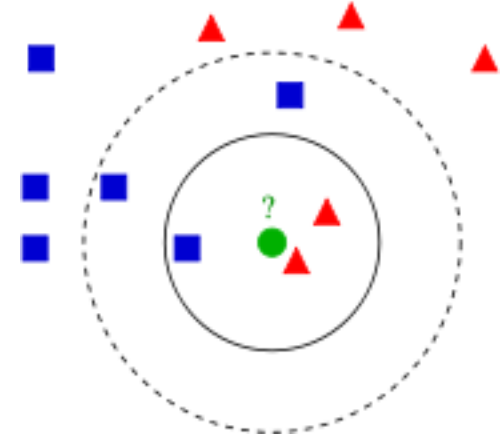
find the distance between the new x' and $x_1, x_2, \dots, x_n$;

x , the closest to x' (***nearest neighbour***) determines the label of x'

Nearest neighbour illustration: Voronoi diagrams



K-nearest neighbour classifier



- Don't just consider one neighbour, but the whole **neighbourhood**
 - Closest k neighbours need to be found
 - Then the majority class is assigned to the instance to be classified
- Let's watch an animation about it

k-nearest neighbor classification

Instance based learner classification

based on the slide by Andrew Moore

(<http://www.cs.cmu.edu/afs/cs/Web/People/awm/tutorials/mbl.html>)

- **Four things make a memory based learner:**
 - **A distance metric**
 - Euclidian/1-norm/hyperbolic/and many more...
 - **How many nearby neighbours to look at?**
 - Nearest/nearest k/all of them
 - **A weighting function (optional)**
 - Would a closer neighbour count more?
 - Kernel function (e.g., $w_k = \exp(-D(x_k, x_{\text{query}})^2 / K_w^2)$)
 - **How to fit with the local points?**
 - Predict nearest neighbour's class/majority vote/use some pretty nasty formula

The best k-NN classifier

- ...is what works best for your data
- What is the best choice of k there?
- What distance measure to use?
- I.e., how to determine the four ingredients you want most?
- If there is no magic compass to tell you the way...

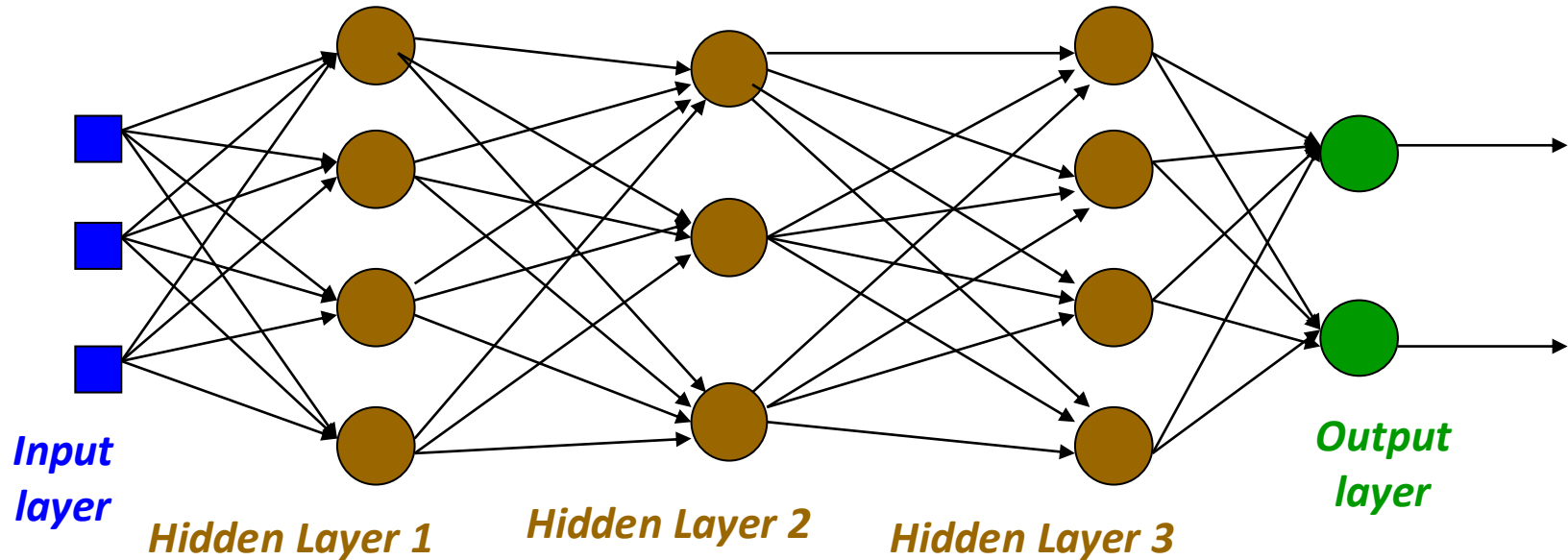


Trial and error will help

(...so trial & error)

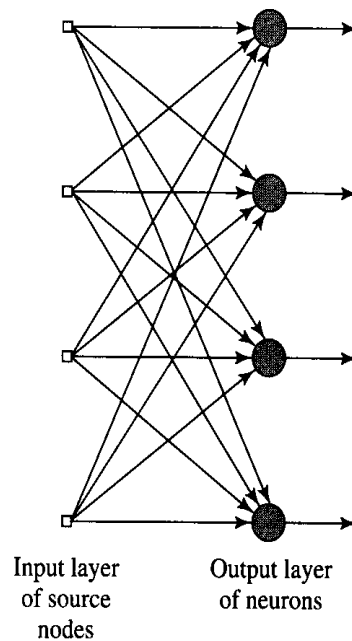
Neural networks

- Processing units (●) and weights (arrows)



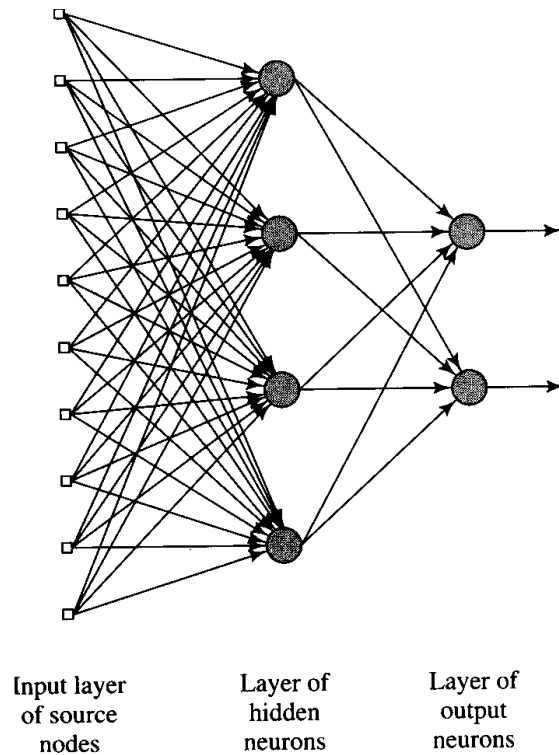
Different types of networks

- Single layer feedforward network



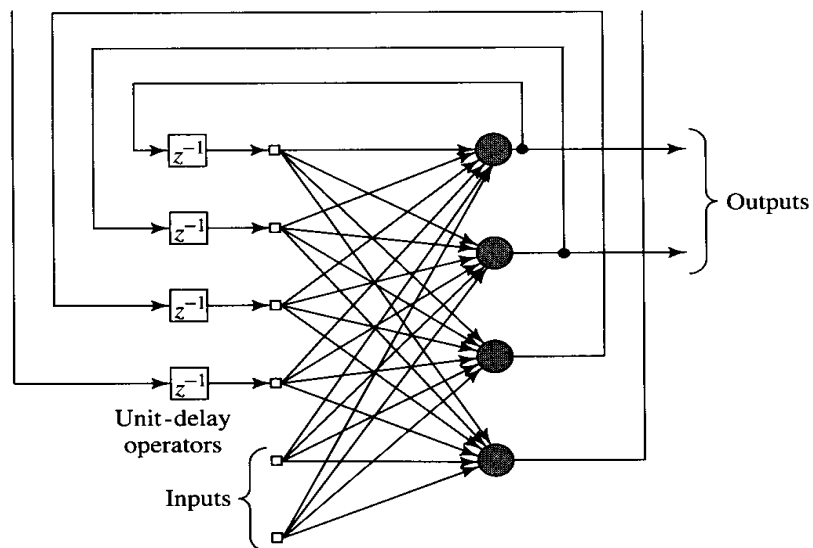
Different types of networks

- Multi layer feedforward network



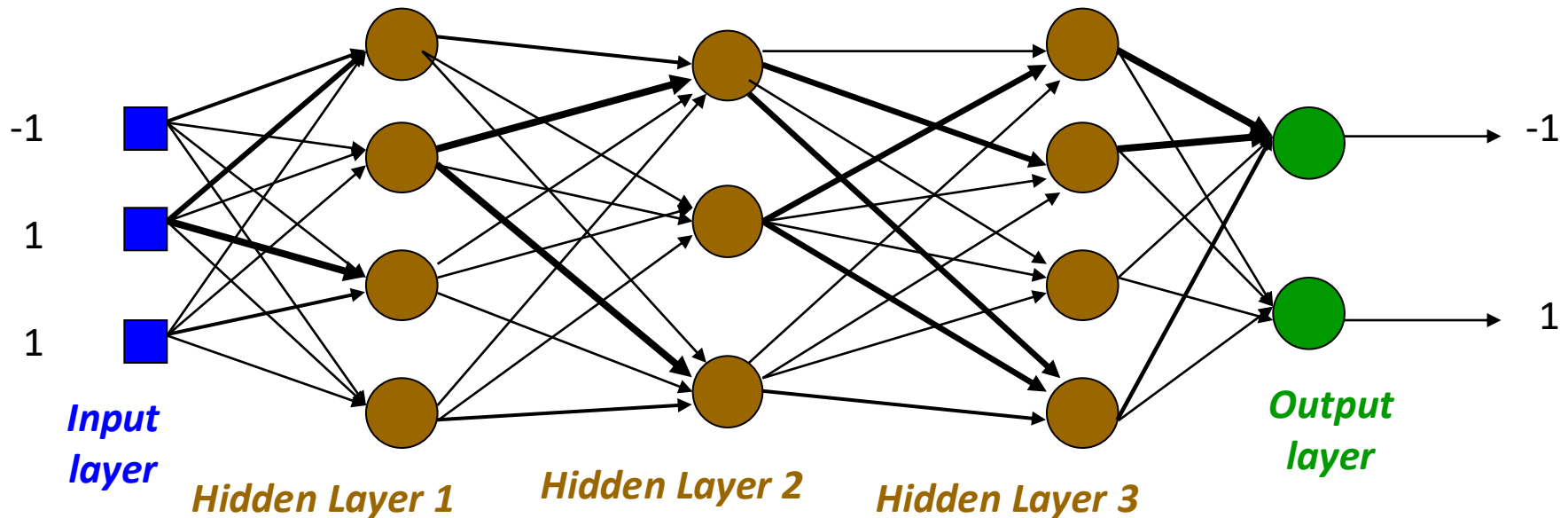
Different types of networks

- Recurrent networks
 - Output is fed back into the network



How do we calculate an answer?

- Propagate activation from the Input towards the Output layer



How do we learn?

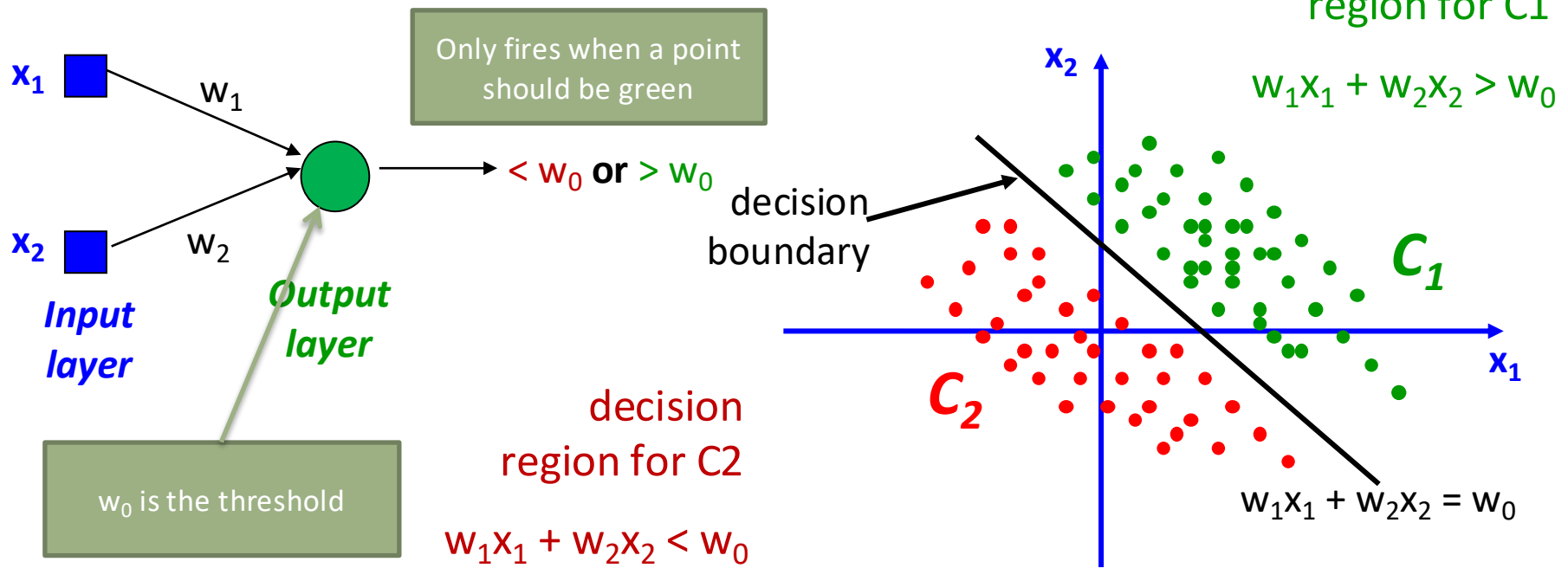
- By adjusting the weights (adaptation)
- Rosenblatt (1958)
 - Perceptron: network of threshold nodes for pattern classification. Perceptron learning rule – first learning algorithm
 - Perceptron convergence theorem:
 - everything that can be represented by a perceptron can be learned

How do we learn?

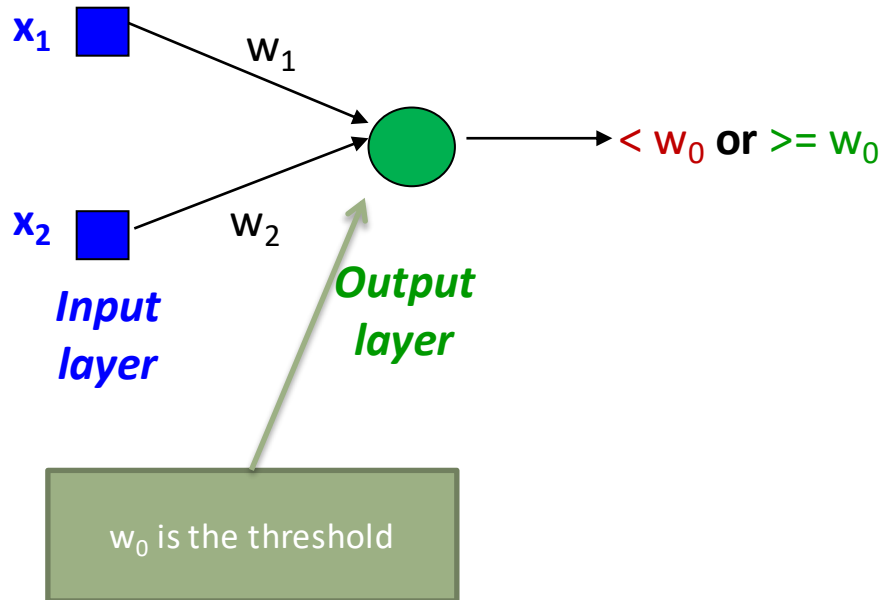
- Simplified main idea:
 - Take an input-output pair from the examples
 - Put the input into the network, calculate the output of the network (the predicted output)
 - Check how different the predicted and actual outputs are
 - Change the weights of the network so that it is less likely to make the same mistake again
 - Do this for all the examples you have

Example (1)

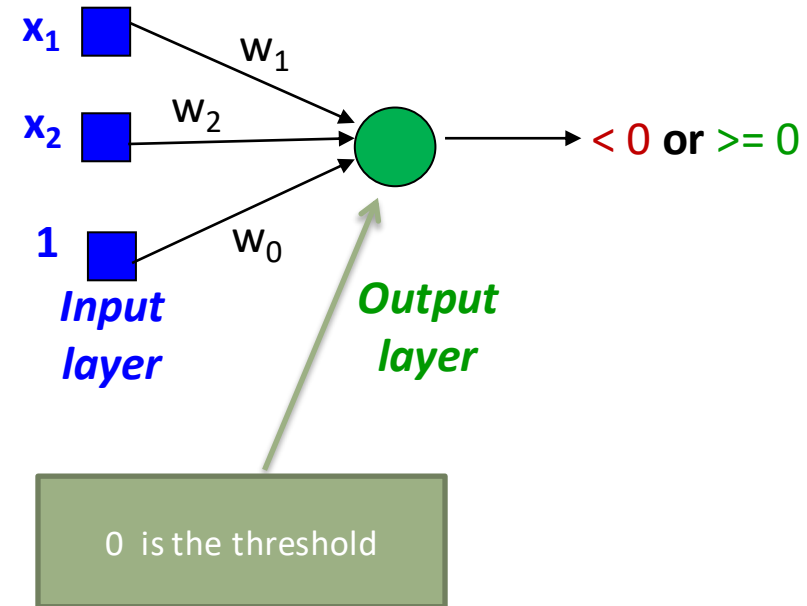
- Problem: separate green from red
- In other words: these dots are the examples.
Now if I give you the coordinates of a new point (x_1, x_2) , tell me if it is green or red.



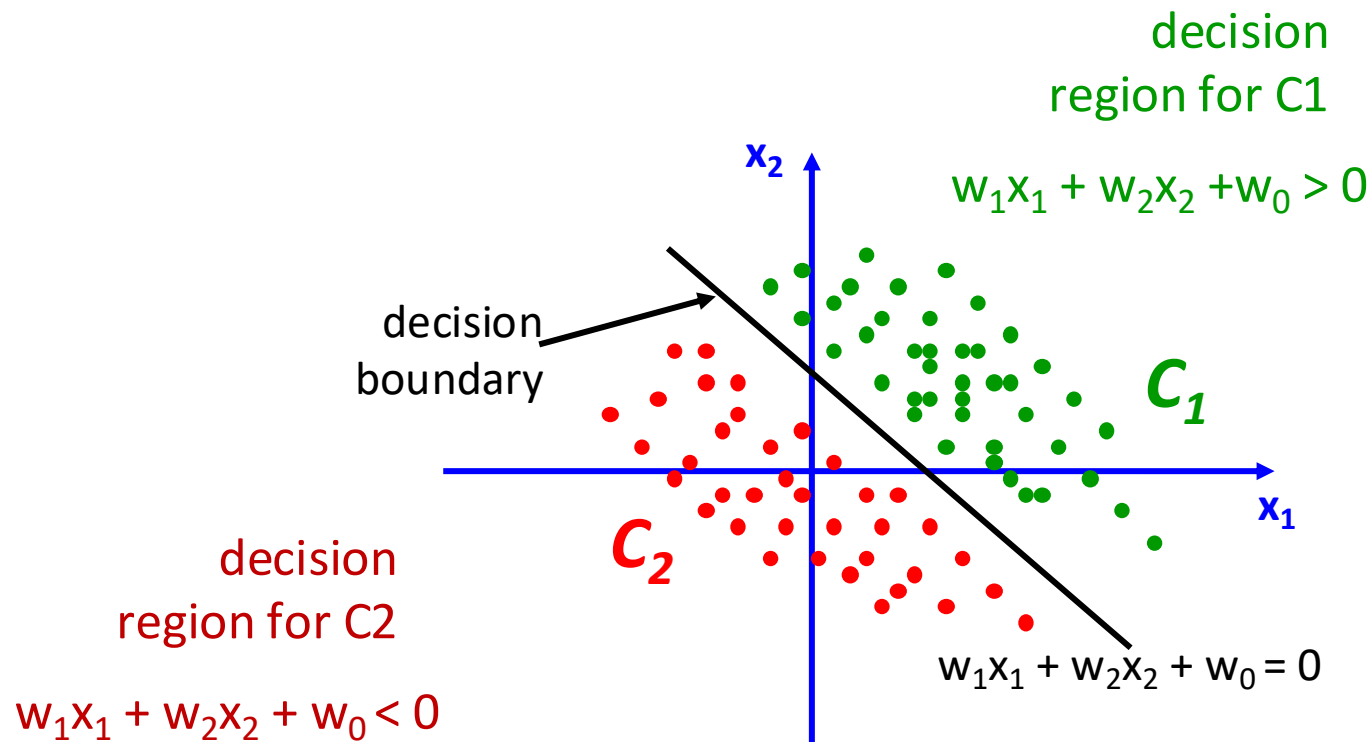
Example (2)



$? =$



Example (3)



Perceptron learning algorithm

Algorithm for learning:

Initialize $\mathbf{w}$ randomly;

While (there are misclassified training examples)

 Select a misclassified example $(\mathbf{x}, y)$

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta d \mathbf{x};$$

End-while

where $\eta > 0$ is a learning rate parameter (step size), and d is either -1 or +1

Algorithm with plain words:

Start with a random line

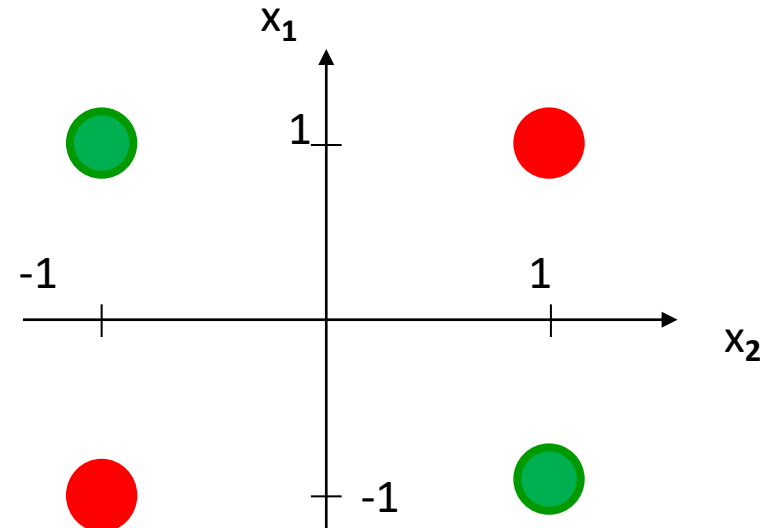
Examine all examples

If you don't get one right, move+turn the line a bit towards that point

But: Disaster!!



- A perceptron can only solve linearly separable problems!
- It can't even separate these reds and greens!
- 1970s → not much funding for ANN research

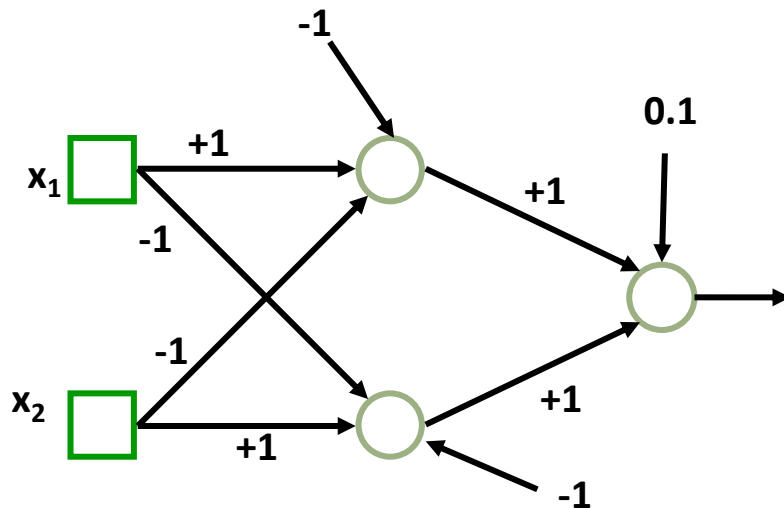
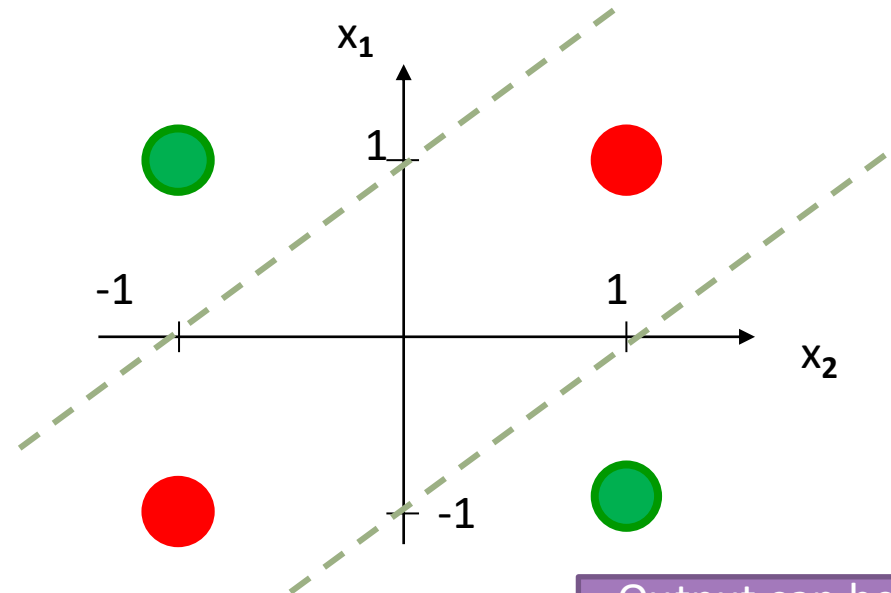


Recovery from disaster

- Renewed enthusiasm and flourish (from 80's)
 - New techniques
 - Backpropagation learning for multi-layer feed forward nets (with non-linear, differentiable node functions)
 - Physics inspired models (Hopfield net, Boltzmann machine, etc.)
 - Unsupervised learning
 - Impressive application (character recognition, speech recognition, text-to-speech transformation, process control, associative memory, etc.)

Example solution for problem

x_1	x_2	$x_1 \text{ xor } x_2$
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1



$$\varphi(v) = \begin{cases} 1 & \text{if } v > 0 \\ -1 & \text{if } v \leq 0 \end{cases}$$

φ is the sign function.

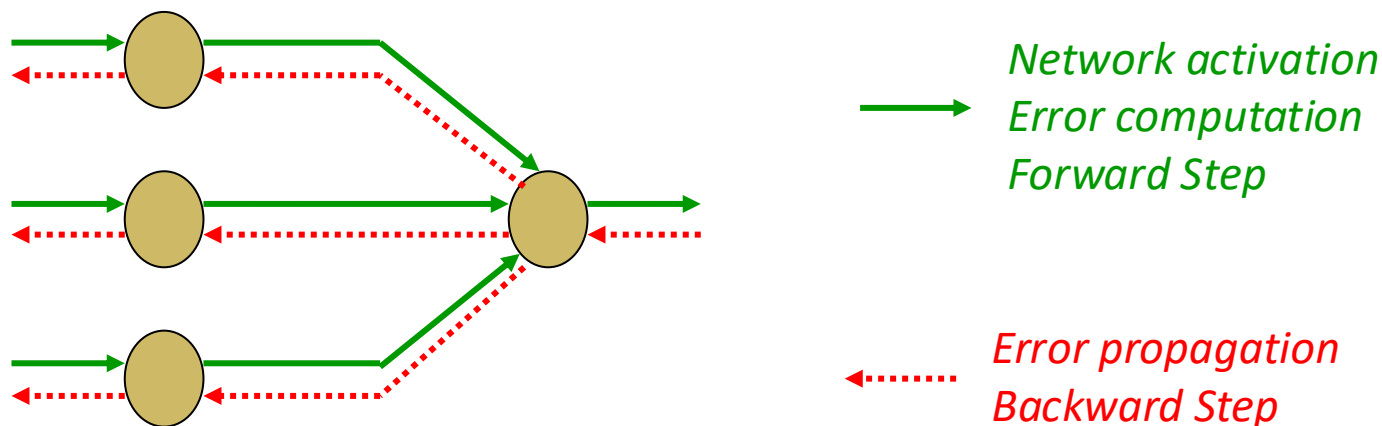
φ is called the activation function

Output can be other than 'on' or 'off', if you choose another activation function!

Learning: backpropagation

(main idea)

- Propagate change backwards, from the output
- Change is proportional to
 - The error the network makes
 - The contribution of a weight to the output



Learning: backpropagation (error)

What is the error we make?

on input i :

$$E(i) = (y_i - f(x_i))^2$$

on all n inputs:

$$E = \sum_{i=0}^n (y_i - f(x_i))^2$$

$$f(x_i) = w_0 + w_1 x_{i1} + \dots + w_k x_{ik}$$

so

$$E = \sum_{i=0}^n (y_i - (w_0 + w_1 x_{i1} + \dots + w_k x_{ik}))^2$$

thus

E is a function of $w_0, w_1, \dots, w_k$

How can we find a minimum of $E(w_0, w_1, \dots, w_k)$?

By applying the gradient descent algorithm ...

Gradient descent

How to find a minimum of a function $f(x,y)$?

1. Start with an arbitrary point (x_0, y_0)
2. Find a direction in which f is decreasing most rapidly

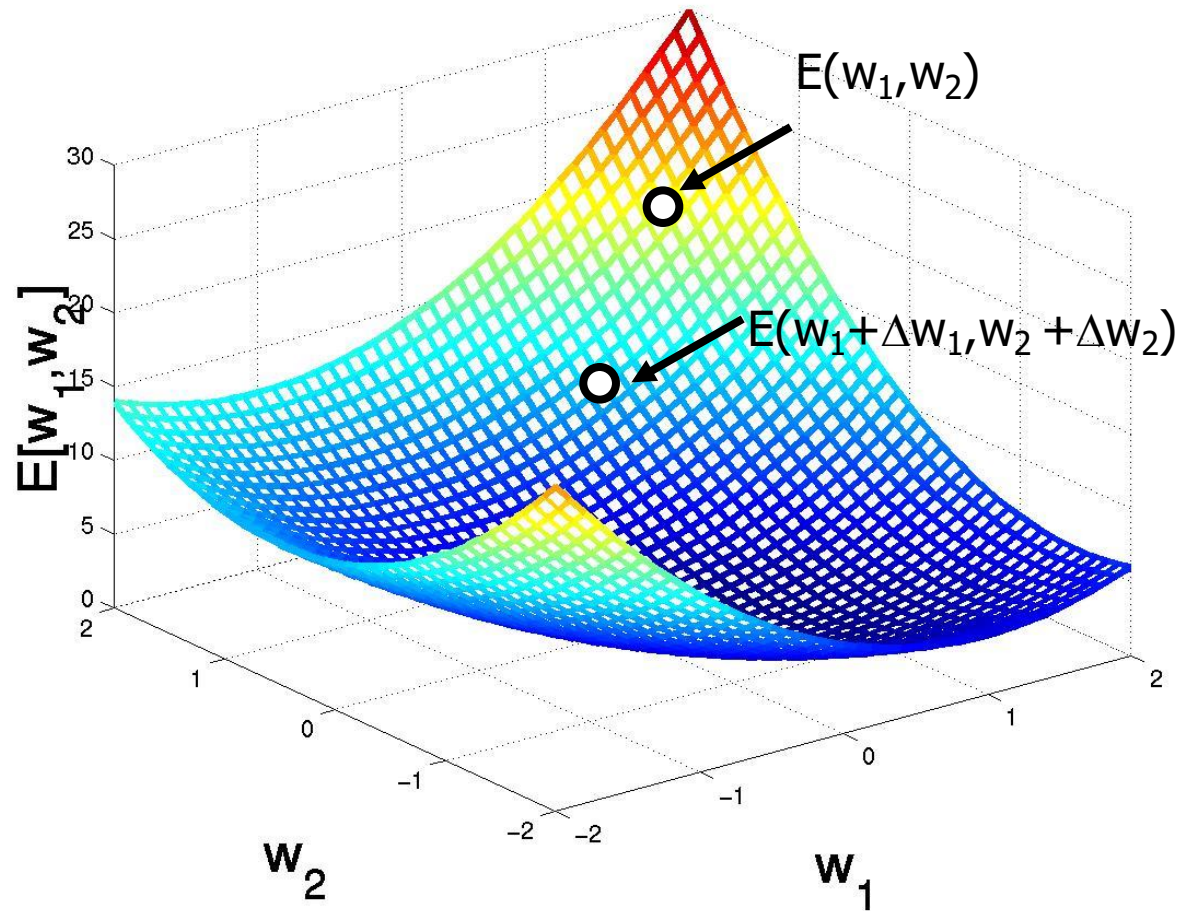
$$-\left[\frac{\partial f(x_0, y_0)}{\partial x}; \frac{\partial f(x_0, y_0)}{\partial y} \right]$$

3. Make a small step in this direction

$$(x_0, y_0) = (x_0, y_0) - \eta \left[\frac{\partial f(x_0, y_0)}{\partial x}; \frac{\partial f(x_0, y_0)}{\partial y} \right]$$

4. Repeat the whole process

Gradient descent



Back Propagation

$\Delta w_{ji} = \eta \delta_j y_i$, where:

$$\delta_j \begin{cases} \varphi'(v_j)(y - y_j) & \text{IF } j \text{ output node} \\ \varphi'(v_j) \sum_{k \text{ in next layer}} \delta_k w_{kj} & \text{IF } j \text{ hidden node} \end{cases}$$

w_{ji} : weight between node i and node j

η : learning rate (constant)

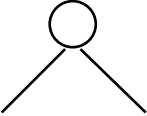
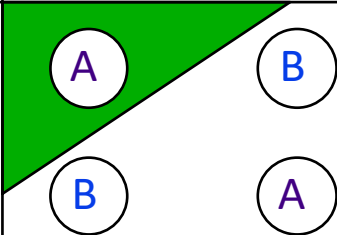
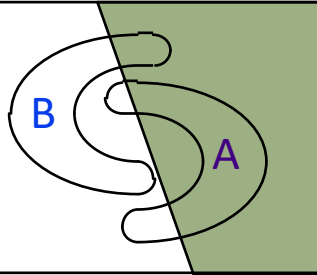
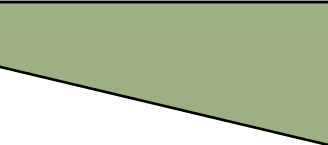
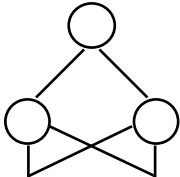
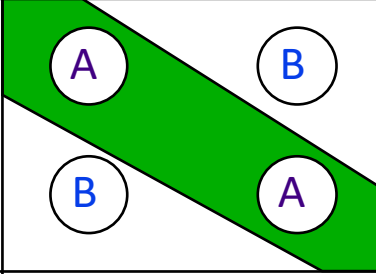
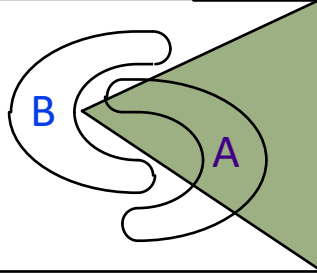
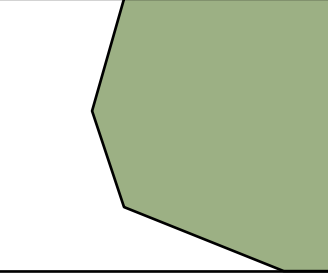
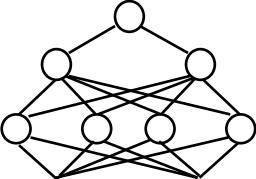
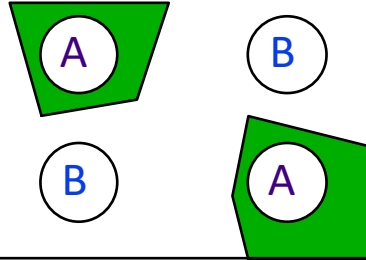
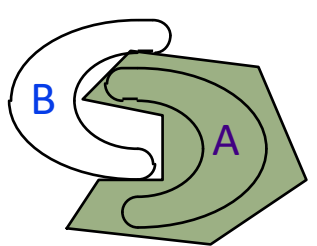
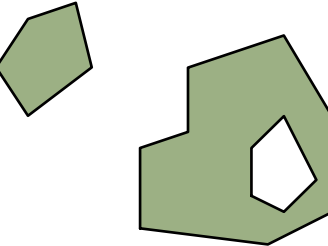
y_j : output of node j

φ : activation function

v_j : activation of node j

y : label of item

So what can they represent?

Structure	Types of Decision Regions	Exclusive-OR Problem	Classes with Meshed regions	Most General Region Shapes
Single-Layer 	Half Plane Bounded By Hyperplane			
Two-Layer 	Convex Open Or Closed Regions			
Three-Layer 	Arbitrary (Complexity Limited by No. of Nodes)			

Association rules: an example

- A shop sells products **a, b, ..., z**
- Clients buy them in collections, e.g., **{a, c}**, **{c, d, z}**, ...
- Each set is called a “transaction” or an “item set”
- What are the most frequent item sets?
- What are the most significant “association rules”: e.g., **{c, g} → {z}**, **{spaghetti, minced meat, tomato} → {basil, cheese}**

Albert Heijn
Middelwaard 93 Zoetermeer
Tel.: 079 - 3213340

	EUR
OPTIMEL YOGH	1,66
AH KARNEMELK	0,72
AH BONUS NR.	
AH MAG YOGH	0,65
CAL SATESAUS	1,64
AH TIVALL	2,49
QUORN	2,39
OPTIM KWARK	2,07
AH TAUGE	0,94
CHIN ROERBAK	1,39
ROERBAKMIX	1,99
ONTBIJTKOEK	1,59
ATK ADV REEP	2,49
ATK ADV REEP	2,49
ATK ADV REEP	2,49
ATK ADV REEP	2,49
ATK ADV REEP	2,49
ATK ADV REEP	2,49
KROEPOEK	0,74
* AIRMILESNR.	
SUBTOTAAL	33,21
AKTIE EFTELINGPUNT	
6 EFTELINGPUNT	
OVER EUR	33,21
TOTAAL	33,21

Not only
one thing!

Products and transactions

{Cheese, beer, diapers}

Product

Pasta	Potato	Onion	Basil	Yoghurt	Milk	Cheese	Beer	Meat	Diapers
1	0	0	1	0	0	1	1	1	0
1	1	1	1	0	0	0	0	0	0
0	0	0	0	0	0	1	1	0	1
1	0	1	0	1	1	0	0	0	1
...	...	...	...	...	...	...	...	...	...

Transaction representation

- Products: thousands
- Transactions: millions
- Data collection: easy



DMT 2010



Examples

- Market basket analysis
 - {*Soy sause* and *tuna*} are often bought together with {*wasabi* and *sushi rice*}
 - Let's place them close together on the shelves (or far?)
- Differential market basket analysis
 - Focus on different patterns in different subsets of the data
 - Compare results to other shops, other demographic group of customers, etc.
 - In this shop, people don't buy {*Soy sauce* and *tuna*} together
 - We need a different strategy
 - Show what similar people bought
 - Recommendation

Support & confidence

“customers who buy Soy sauce & tuna also buy wasabi and sushi rice”

$\{\text{Soy_Sauce, Tuna}\} \Rightarrow \{\text{Wasabi, Sushi_Rice}\}$

support : 0.5% ; confidence : 85%

- **Support** (aka. Coverage) of an itemset:
 - number/ratio/percentage of instances occurring in the data
 - “The above four items occur together in 0.5% of the transactions.”
- **Confidence** (aka. Accuracy) of a rule:
 - proportion of the number of instances to which the rule applies
 - “When *Soy sauce* and *tuna* are bought together, in 85% of the cases *wasabi* and *sushi rice* are also bought.”

Formal definitions

items:	$I = \{i_1, i_2, \dots, i_n\}$
itemset:	a collection of items
k -itemset:	an itemset with k elements
transaction:	a collection of items
D :	a fixed list of transactions (a table)

items are binary attributes (!)

transactions = records with n binary fields

“an item set S is included in a transaction T ”:

$$S=\{A, B\}; T=\{A, B, C, D\}; S \subset T$$

Formal definitions

An association rule: $X \Rightarrow Y$
where X and Y are disjoint itemsets

The rule $X \Rightarrow Y$ has **support** s (in D) if
 $s\%$ of transactions from D contain $X \cup Y$

The rule $X \Rightarrow Y$ has **confidence** c if
 $c\%$ of transactions from D that contain X also contain Y

$$\text{confidence}(X \Rightarrow Y) = \text{support}(X \cup Y) / \text{support}(X)$$

It's the weather again...

If humidity=normal & play=yes then windy=false

Table 1.2 **The weather data.**

outlook	temperature	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

14 transactions

Support =
 $4/14 = 28.5\%$

Confidence =
 $4/6 = 66.6\%$

Issues with association rules

- For 1000 items there are 2^{1000} itemsets
- Each k -itemset gives 2^k rules
- Huge data sets (terabytes of data)

But:

Only rules with **high support and confidence** might be interesting...

Conclusion:

*Focus search on rules with **high support & confidence***

Searching for rules...

- Split the search process into two steps:
 - I. Find all “frequent” itemsets and their support
 - II. For each frequent itemset find all rules that have high confidence

*(an item set is said to be frequent if it occurs in transactions in D with **frequency higher than s** (e.g., $s=3\%$))*
- Consider that
 - If $Z=X \cup Y$ is a frequent itemset then so is X
 - The rule $X \Rightarrow Y$ has support = s and confidence = $\text{support}(X \cup Y) / \text{support}(X)$

I. Finding frequent itemsets

- L_k = all frequent sets with k items
- C_k = a set of “potentially frequent” sets with k items

find L_1

repeat

1) given L_k generate C_{k+1}

2) scan the data D and find all
frequent sets in C_{k+1} ; they will form L_{k+1}

until L_{k+1} is empty

return $L_1, L_2, \dots$

Generating candidate sets

- Given L_k = all frequent sets with k items
- **Find** C_{k+1} = a set of “potentially frequent” $(k+1)$ -sets
- ***A subset of a frequent set is also frequent***, thus
 - sets from C_{k+1} must be supersets of sets from L_k
 - all k -element subsets of elements of C_{k+1} must be in L_k

Example:

$L_3 = \{\{\text{Soy, Tuna, Wasabi}\}, \{\text{Soy, Tuna, Rice}\}, \{\text{Soy, Wasabi, Rice}\}, \{\text{Soy, Wasabi, Sake}\}, \{\text{Tuna, Wasabi, Rice}\}\};$

C_4 could be $\{\{\text{Soy, Tuna, Wasabi, Rice}\}, \{\text{Soy, Tuna, Wasabi, Sake}\}, \{\text{Soy, Wasabi, Rice, Sake}\}\}$, correct?

No, $\{\text{Soy, Tuna, Wasabi, Sake}\}$ can be removed (because of $\{\text{Tuna, Wasabi, Sake}\}$) does not occur in L_3

No, $\{\text{Soy, Wasabi, Rice, Sake}\}$ can be removed (because of $\{\text{Wasabi, Rice, Sake}\}$) does not occur in L_3

Formally, from L_k to C_{k+1}

1. $C_{k+1} = \emptyset$;
2. Insert to C_{k+1} all sets of the form $p \cup q$,
where $p, q \in L_k$, and $|p \cup q| = k+1$
3. Remove from C_{k+1} all sets that have a k -
subset that does not belong to L_k

From C_k to L_k

For all transactions $t \in D$ do:

//find all itemsets from C_k which are included in t :

$C_t = \text{subset}(C_k, t)$;

for all $c \in C_t$ do:

$c.\text{count}++$;

end

end

$L_k = \{c \in C_k \mid c.\text{count} > s\}$

$\{\text{Soy, Tuna, Wasabi}\}.\text{count} = 5\% > 3\%$

$\{\text{Soy, Wasabi, Sake}\}.\text{count} = 7\% > 3\%$

II. Generating rules

- Let L be a frequent itemset, $A \subset L$.
- If the confidence of $(L-A) \Rightarrow A$ is c then for any $B \subset A$ the confidence of $(L-B) \Rightarrow B$ is at least c .

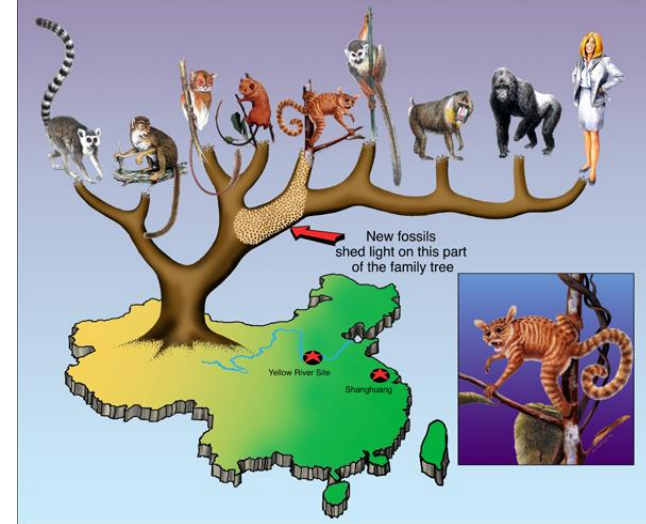
$$\text{Confidence}(\{\text{Soy}, \text{Tuna}\} \Rightarrow \{\text{Wasabi}, \text{Rice}\}) \leq \text{Confidence}(\{\text{Soy}, \text{Tuna}, \text{Rice}\} \Rightarrow \{\text{Wasabi}\})$$

- Therefore, rules can be generated incrementally, starting with rules with 1-itemset conclusions, then 2-itemset conclusions, etc.

Experiments & results

- Find them in the paper: Zaki et al. (1997) *New Algorithms for Fast Discovery of Association Rules*. (Canvas)
-
- Scaling and speed
 - The cpu-time seems to increase linearly (from a few seconds to a few minutes)
 - Further speed-up can be obtained by sampling
 - Sampling useful only for relatively big support s .

Extensions



- What to do with very similar items?
 - Pizza hawaii, mushroom pizza, pizza margherita (?=>?) amstel, heineken
- Now, how to deal with **taxonomies** of items?
- How to reduce the number of generated rules?
- How to define the concept of an “interesting rule”?
- How to deal with multiple items? (e.g., 3 bottles of beer)

By now, many papers have been published on these and other issues ...

Summary

- Today we learned about
 - Classification algorithms (take 2):
 - Instance-based learning
 - Neural Networks
 - Association rules